MINOR PROJECT REPORT

Traffic Light Controller Using Verilog

Project components: Timer-Based Controller and Sensor-Based Controller

STUDENT NAME	SRIVIDYA A B
BATCH	OCTOBER

Platform: EDA Playground (Icarus Verilog and EPWave)

1. Aim and Introduction

Aim: To design and simulate two traffic light controllers using Verilog: one that changes signals after fixed counter intervals and another that responds to sensor inputs.

A traffic light controller decides which direction receives a green, yellow, or red signal. In this project, each controller is described in Verilog and tested using a separate testbench. Simulation waveforms are used to observe how the outputs change.

2. Tools Used

EDA Playground (browser-based editor), Icarus Verilog 12.0 (simulator), and EPWave (waveform viewer).

No physical hardware was used for these simulations.

3. Basic Design Approach

The timer-based controller uses six states (S0 to S5) and a counter. The sensor-based controller uses four states (S0 to S3), with SNS1 and SNS2 controlling two state transitions. Both designs have a clock and reset input.

4. Working Principle:

The timer-based traffic light controller uses six states (S0–S5). A counter controls how long the controller stays in each state. After the required count, the controller moves to the next state.

The sensor-based traffic light controller uses four states (S0–S3). The sensor inputs SNS1 and SNS2 determine when the traffic lights change. The controller uses green, yellow and red signals to manage traffic flow.

5 a. CODE

Timer-Based Controller – Design Code:

```
`timescale 1ns / 1ps
module Traffic_Light_Timer(
    input wire clk,
    input wire clr,
    output reg [5:0] lights
);
reg [2:0] state;
reg [3:0] count;
parameter S0=3'b000, S1=3'b001, S2=3'b010,
          S3=3'b011, S4=3'b100, S5=3'b101;
parameter SEC5=4'b1111;
parameter SEC1=4'b0011;
always @(posedge clk or posedge clr)
begin
    if (clr == 1)
    begin
        state <= S0;
        count <= 0;
    end
    else
    begin
        case (state)
            S0: if (count < SEC5) count <= count + 1;
                else begin state <= S1; count <= 0; end
            S1: if (count < SEC1) count <= count + 1;
                else begin state <= S2; count <= 0; end
            S2: if (count < SEC1) count <= count + 1;
                else begin state <= S3; count <= 0; end
            S3: if (count < SEC5) count <= count + 1;
                else begin state <= S4; count <= 0; end
            S4: if (count < SEC1) count <= count + 1;
                else begin state <= S5; count <= 0; end
            S5: if (count < SEC1) count <= count + 1;
```

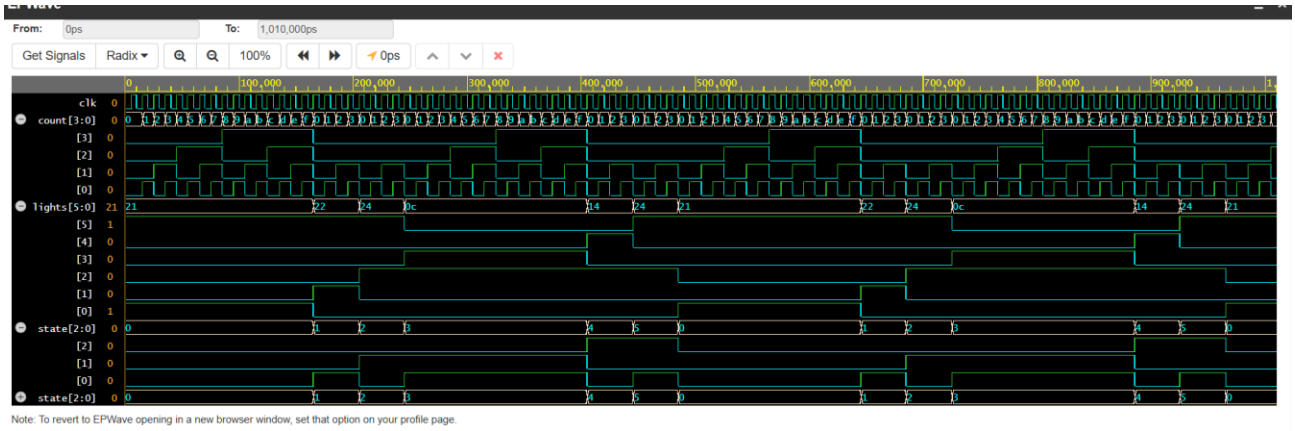
```
                else begin state <= S0; count <= 0; end
            default: begin state <= S0; count <= 0; end
        endcase
    end
end
always @(*)
begin
    case (state)
        S0: lights = 6'b100001;
        S1: lights = 6'b100010;
        S2: lights = 6'b100100;
        S3: lights = 6'b001100;
        S4: lights = 6'b010100;
        S5: lights = 6'b100100;
        default: lights = 6'b100001;
    endcase
end
endmodule
```

Timer-Based Controller – Testbench:

```
`timescale 1ns / 1ps
module testbench;
reg clk;
reg clr;
wire [5:0] lights;
Traffic_Light_Timer t1(clk, clr, lights);
initial
begin
    clk = 0;
    forever #5 clk = ~clk;
end
initial
begin
    clr = 1;
    #10 clr = 0;
    #1000 $finish;
end
initial
begin
    $dumpfile("dump.vcd");
    $dumpvars(0, testbench);
end
endmodule
```

5 b. Timer-Based Simulation Result

The waveform shows the clock, a counter that increases and resets, and the state sequence 0 → 1 → 2 → 3 → 4 → 5 → 0. The six-bit light output changes with each state. The values displayed in EPWave use hexadecimal notation.



Observation:

The simulation shows that the controller changes through six states, from S0 to S5, and returns to S0. The counter increases during each state and resets when the state changes. The traffic light outputs change according to the programmed state values.

Sensor-Based Controller – Design Code:

```
`timescale 1ns / 1ps
module Traffic_Light_Sensor(
    input clk,
    input clr,
    input SNS1,
    input SNS2,
    output reg [1:0] TL1, TL2
);
reg [1:0] state, nextstate;
parameter S0=2'b00, S1=2'b01,
          S2=2'b10, S3=2'b11;
parameter green=2'b00, yellow=2'b01, red=2'b10;
always @(posedge clk or posedge clr)
begin
    if (clr) state <= S0;
    else state <= nextstate;
end
always @(*)
begin
    case (state)
        S0: if (SNS2) nextstate = S1;
            else nextstate = S0;
        S1: nextstate = S2;
        S2: if (SNS1) nextstate = S3;
            else nextstate = S2;
        S3: nextstate = S0;
        default: nextstate = S0;
    endcase
end
always @(*)
begin
    if (state == S0)
    begin TL1 = green; TL2 = red; end
    else if (state == S1)
    begin TL1 = yellow; TL2 = red; end
```

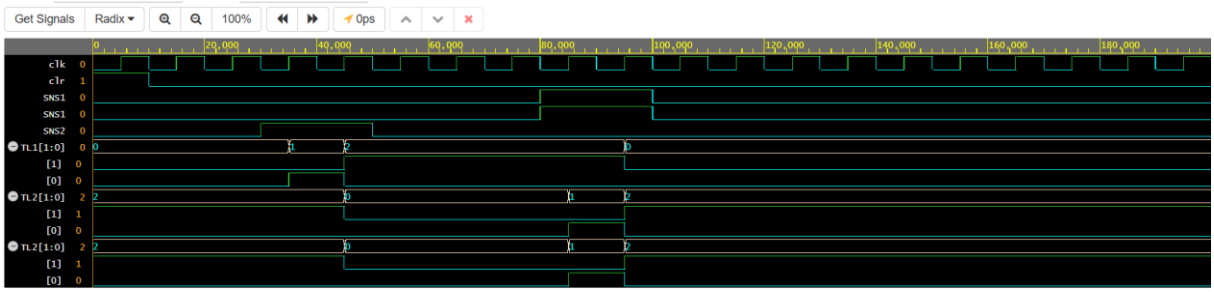
```
    else if (state == S2)
    begin TL1 = red; TL2 = green; end
    else
    begin TL1 = red; TL2 = yellow; end
end
endmodule
```

Sensor-Based Controller — Testbench

```
`timescale 1ns / 1ps
module testbench;
reg clk, clr, SNS1, SNS2;
wire [1:0] TL1, TL2;
Traffic_Light_Sensor t1(clk, clr, SNS1, SNS2, TL1, TL2);
initial
begin
    clk = 0;
    forever #5 clk = ~clk;
end
initial
begin
    clr = 1;
    SNS1 = 0;
    SNS2 = 0;
    #10 clr = 0;
    #20 SNS2 = 1;
    #20 SNS2 = 0;
    #30 SNS1 = 1;
    #20 SNS1 = 0;
    #100 $finish;
end
initial
begin
    $dumpfile("dump.vcd");
    $dumpvars(0, testbench);
end end module
```

5.c. Sensor-Based Simulation Result

The waveform shows sensor inputs and two traffic light outputs. The expected light sequence is TL1/TL2: green/red → yellow/red → red/green → red/yellow → green/red.



Observation:

Initially, Traffic Light 1 is green and Traffic Light 2 is red. When SNS2 is activated, Traffic Light 1 changes to yellow, followed by red, while Traffic Light 2 becomes green. When SNS1 is activated, Traffic Light 2 changes to yellow before the controller returns to its initial state.

6. Observations and Conclusion

Timer-based design: The controller cycles through six states, with the counter determining when the next state begins. Sensor-based design: The controller waits for SNS2 before leaving S0 and waits for SNS1 before leaving S2. S1 and S3 are intermediate yellow-light states.

Result: Both Verilog designs were simulated using Icarus Verilog, and their output changes were observed in EPWave. The included screenshots document the observed behavior.

Conclusion: The project demonstrates how a finite-state machine can control traffic signals using either counter-based timing or sensor-driven transitions.